Página de inicio: cambios pendientes

Qué falta respecto al prototipo y a la guía, y dónde va cada cambio

Archivos: `lib/main.dart`, `lib/screens/home.dart` y archivos nuevos · 20 de octubre de 2026 · Paso 10 de la guía de construcción (Opción A)

Tu versión funciona y conserva bien la navegación (`_methods`, `_push`, `_pushWithCopy`). El problema es que es un rediseño visual de la página anterior, no la página del prototipo. Este documento lista lo que hay que cambiar, en qué archivo va cada cosa y el código de referencia.

> **Sobre los colores:** hay que dejar de escribirlos dentro de los widgets y pasarlos a `AppColors` con nombres de uso (sección 3). Así cualquier cambio de color se hace en un solo archivo y toda la página se actualiza.

1. Resumen

Qué	Estado	Qué hacer
Navegación a cada pantalla (`_methods`, `_push`, `_pushWithCopy`)	Bien	Conservar tal cual.
Lista horizontal de algoritmos al final	Bien, a medias	Conservar; añadir entrada escalonada, efecto hover/clic y arrastre con mouse en web (sección 5).
Tema oscuro forzado en `main.dart`	Cambiar	Afecta a toda la app, no solo al inicio. Quitarlo (sección 2).
Fuente `'.SF Pro Text'`	Cambiar	Es la fuente privada del sistema de Apple; no se debe referenciar así y en web/Android no existe. Quitarla.
Colores escritos dentro de los widgets	Cambiar	Pasarlos a `AppColors` con nombres (sección 3).
Acordeón de conceptos	Reemplazar	Tarjetas que se deslizan con el scroll, alternando izquierda/derecha, con imagen y botón de YouTube (sección 4).
Títulos de sección sin animación	Falta	Aparición única al entrar en pantalla (sección 5).
Constantes sin usar, `withOpacity`	Limpiar	Borrar las constantes que no se usan; `withOpacity` está obsoleto, usar `withValues(alpha: …)`.

2. `main.dart`: no cambiar el tema de toda la app

`lib/main.dart → MaterialApp(...)`

Con `themeMode: ThemeMode.dark` y un `theme` también oscuro, **todas** las pantallas pasan a fondo negro: el editor, Johnson, Asignación, las matrices y los ordenamientos. Esas pantallas están pensadas sobre fondo claro; por ejemplo, las etiquetas de los nodos ficticios de Asignación se dibujan en negro y desaparecen sobre el negro.

> **Regla:** el aspecto del inicio se define **dentro de** `HomeScreen`. `main.dart` no debe cambiar el tema global en esta tarea.

Dejar `main.dart` como estaba (sin `themeMode`, sin `fontFamily`, sin colores Apple):

```
return MaterialApp(
  title: 'Graph Editor',
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
    useMaterial3: true,
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo, brightness: Brightness.dark),
    useMaterial3: true,
  ),
  home: const HomeScreen(),
  debugShowCheckedModeBanner: false,
);
```

Si el equipo decide que el inicio sea oscuro, se hace solo en esa pantalla: `Scaffold(backgroundColor: AppColors.homeBackground, ...)`, y los textos toman su color de `AppColors`.

3. Colores: de los widgets a `AppColors`

`lib/theme/app_colors.dart → al final de la clase AppColors`

Añadir este bloque. Los valores de ejemplo son los que usas hoy, así la página se ve igual que ahora; para cambiar un color basta con editar su valor aquí.

```
  // ---------- Inicio ----------
  static const Color homeBackground    = Color(0xFF000000); // fondo de la página
  static const Color homeSurface       = Color(0xFF1C1C1E); // tarjetas
  static const Color homeSurfaceAlt    = Color(0xFF2C2C2E); // ícono de la tarjeta, imagen
  static const Color homeText          = Color(0xFFFFFFFF); // títulos
  static const Color homeTextSecondary = Color(0xFF8E8E93); // textos largos
  static const Color homeAccent        = Color(0xFFFF9F0A); // botón Abrir, número de tarjeta
  static const Color homeOnAccent      = Color(0xFF000000); // texto sobre el acento
  static const Color homeVideo         = Color(0xFFB91C1C); // botón "Ver en YouTube"
```

`lib/screens/home.dart → _HomeScreenState`

Borrar `_black`, `_darkGray`, `_gray`, `_lightGray`, `_orange`, `_white` y `_secondary` (cuatro de ellas ni se usan). En `_ConceptTile` y `_MethodCard` reemplazar cada `Color(0xFF...)` por su token:

Hoy en el código	Reemplazar por
`Color(0xFF000000)` como fondo	`AppColors.homeBackground`
`Color(0xFF1C1C1E)`	`AppColors.homeSurface`
`Color(0xFF2C2C2E)`, `Color(0xFF3A3A3C)`	`AppColors.homeSurfaceAlt`
`Color(0xFFFFFFFF)` en textos	`AppColors.homeText`
`Color(0xFF8E8E93)`	`AppColors.homeTextSecondary`

`Color(0xFFFF9F0A)`	`AppColors.homeAccent`
`Color(0xFF000000)` sobre naranja	`AppColors.homeOnAccent`
`.withOpacity(0.4)`	`.withValues(alpha: 0.4)`

Comprobación: al terminar, buscar `Color(0x` en `home.dart` no debe dar ningún resultado.

4. Conceptos: tarjetas que se deslizan con el scroll

El acordeón (`_ConceptTile`, `_openConcept`) se elimina. Cada concepto pasa a ser una tarjeta siempre abierta con **número, título, texto, imagen y botón «Ver en YouTube»**. Su movimiento depende del scroll, no de un temporizador:

- Empieza a entrar cuando su borde superior aparece por abajo de la pantalla.
- Se detiene cuando la tarjeta se ve **completa**.
- Si el usuario sube, la tarjeta vuelve a salir por el mismo lado.
- Alternan: 01 y 03 desde la **izquierda**, 02 y 04 desde la **derecha**. La imagen va del lado por el que entra la tarjeta.

Ver el comportamiento en la pantalla «Inicio» del prototipo.

4.1 Paquetes

`pubspec.yaml → dependencies`

```
  url_launcher: ^6.3.0          # abrir el enlace de YouTube
  visibility_detector: ^0.4.0   # apariciones únicas (sección 5)
```

Y declarar las imágenes en `flutter: assets:` → `- assets/images/conceptos/`.

4.2 Datos de cada concepto

`lib/screens/home.dart → class _Concept y const _concepts`

```
class _Concept {
  final String title;
  final String text;
  final String image;    // ruta del asset
  final String videoUrl; // enlace de YouTube
  const _Concept(this.title, this.text, this.image, this.videoUrl);
}

const _concepts = <_Concept>[
  _Concept('¿Qué es un grafo?', '...texto actual...',
      'assets/images/conceptos/grafo.png',
      'https://www.youtube.com/results?search_query=que+es+un+grafo'),
  _Concept('¿Para qué sirven?', '...', 'assets/images/conceptos/usos.png',
      'https://www.youtube.com/results?search_query=aplicaciones+teoria+de+grafos'),
  _Concept('Concepto matemático de un grafo', '...', 'assets/images/conceptos/formal.png',
      'https://www.youtube.com/results?search_query=definicion+formal+grafo'),
  _Concept('Tipos de grafos', '...', 'assets/images/conceptos/tipos.png',
      'https://www.youtube.com/results?search_query=tipos+de+grafos'),
];
```

Los enlaces son búsquedas de YouTube como marcador; se cambian por los videos elegidos. Las imágenes pueden ser capturas de la app mientras no haya otras.

4.3 Widget nuevo: `ScrollSlideCard`

lib/screens/home/scroll_slide_card.dart (archivo nuevo)

```
import 'package:flutter/material.dart';

/// Desliza [child] desde un lado según el scroll.
/// progreso 0 = el borde superior entra por abajo; 1 = la tarjeta se ve completa.
class ScrollSlideCard extends StatefulWidget {
  final Widget child;
  final bool fromLeft;
  const ScrollSlideCard({super.key, required this.child, required this.fromLeft});

  @override
  State<ScrollSlideCard> createState() => _ScrollSlideCardState();
}

class _ScrollSlideCardState extends State<ScrollSlideCard> {
  final _progress = ValueNotifier<double>(1);
  ScrollPosition? _position;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();
    _position?.removeListener(_measure);
    _position = Scrollable.maybeOf(context)?.position;
    _position?.addListener(_measure);
    WidgetsBinding.instance.addPostFrameCallback((_) => _measure());
  }

  // Se mide en el listener del scroll y después del frame, nunca dentro de build().
  void _measure() {
    if (!mounted) return;
    final box = context.findRenderObject() as RenderBox?;
    final viewport = Scrollable.maybeOf(context)?.context.findRenderObject() as RenderBox?;
    if (box == null || viewport == null || !box.attached) return;
    final top = box.localToGlobal(Offset.zero, ancestor: viewport).dy;
    final vh = viewport.size.height;
    final span = box.size.height < vh ? box.size.height : vh;
    _progress.value = ((vh - top) / span).clamp(0.0, 1.0);
  }

  @override
  void dispose() {
    _position?.removeListener(_measure);
    _progress.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    // Movimiento reducido activado en el dispositivo: sin deslizamiento.
    if (MediaQuery.disableAnimationsOf(context)) return widget.child;
    final width = MediaQuery.sizeOf(context).width;
    return ValueListenableBuilder<double>(
      valueListenable: _progress,
      child: widget.child,
      builder: (context, p, child) {
        final e = Curves.easeOutCubic.transform(p);
        final dx = (widget.fromLeft ? -1 : 1) * (1 - e) * width * 0.75;
        return Opacity(
```

```
          opacity: 0.1 + 0.9 * e,
          child: Transform.translate(offset: Offset(dx, 0), child: child),
        );
      },
    );
  }
}
```

4.4 La tarjeta de concepto

`lib/screens/home.dart → reemplaza a class _ConceptTile`

```
class _ConceptCard extends StatelessWidget {
  final _Concept concept;
  final int index; // 0, 1, 2, 3
  const _ConceptCard({required this.concept, required this.index});

  bool get _fromLeft => index.isEven;

  @override
  Widget build(BuildContext context) {
    final image = ClipRRect(
      borderRadius: BorderRadius.circular(22),
      child: AspectRatio(
        aspectRatio: 16 / 10,
        child: Image.asset(concept.image, fit: BoxFit.cover),
      ),
    );

    final text = Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        _NumberBadge(index + 1),               // "01", fondo AppColors.homeAccent
        const SizedBox(height: 12),
        Text(concept.title, style: const TextStyle(
            fontSize: 30, fontWeight: FontWeight.w600, color: AppColors.homeText)),
        const SizedBox(height: 10),
        Text(concept.text, style: const TextStyle(
            fontSize: 16, height: 1.6, color: AppColors.homeTextSecondary)),
        const SizedBox(height: 16),
        _YoutubeButton(url: concept.videoUrl),
      ],
    );

    return ScrollSlideCard(
      fromLeft: _fromLeft,
      child: Align(
        alignment: _fromLeft ? Alignment.centerLeft : Alignment.centerRight,
        child: ConstrainedBox(
          constraints: const BoxConstraints(maxWidth: 1000),
          child: Container(
            padding: const EdgeInsets.all(20),
            decoration: BoxDecoration(
              color: AppColors.homeSurface,
              borderRadius: BorderRadius.circular(32),
            ),
            child: LayoutBuilder(builder: (context, c) {
              if (c.maxWidth < 700) {           // teléfono: imagen arriba, texto abajo
                return Column(children: [image, const SizedBox(height: 20), text]);
              }
```

```
              final parts = [
                Expanded(child: image),
                const SizedBox(width: 28),
                Expanded(child: text),
              ];
              // La imagen va del lado por el que entra la tarjeta.
              return Row(children: _fromLeft ? parts : parts.reversed.toList());
            }),
          ),
        ),
      ),
    );
  }
}
```

Botón de YouTube (mismo archivo):

```
class _YoutubeButton extends StatelessWidget {
  final String url;
  const _YoutubeButton({required this.url});

  @override
  Widget build(BuildContext context) {
    return Pressable(
      onTap: () => launchUrl(Uri.parse(url), mode: LaunchMode.externalApplication),
      child: Container(
        height: 48,
        padding: const EdgeInsets.symmetric(horizontal: 18),
        decoration: const ShapeDecoration(color: AppColors.homeVideo, shape: StadiumBorder()),
        child: const Row(mainAxisSize: MainAxisSize.min, children: [
          Icon(Icons.play_circle_fill, color: Colors.white),
          SizedBox(width: 8),
          Text('Ver en YouTube', style: TextStyle(color: Colors.white, fontWeight: FontWeight.w600)),
        ]),
      ),
    );
  }
}
```

Número de la tarjeta (mismo archivo):

```
class _NumberBadge extends StatelessWidget {
  final int n;
  const _NumberBadge(this.n);

  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      decoration: const ShapeDecoration(color: AppColors.homeAccent, shape: StadiumBorder()),
      child: Text(n.toString().padLeft(2, '0'),
          style: const TextStyle(color: AppColors.homeOnAccent, fontWeight: FontWeight.w600)),
    );
  }
}
```

4.5 Dónde se usan

`lib/screens/home.dart → imports nuevos`

```
import 'package:url_launcher/url_launcher.dart';
import 'package:graph_maker_app_2/theme/app_colors.dart';
import 'package:graph_maker_app_2/widgets/ui/pressable.dart';
import 'package:graph_maker_app_2/screens/home/scroll_slide_card.dart';
import 'package:graph_maker_app_2/screens/home/reveal_once.dart';
```

`lib/screens/home.dart → build() de _HomeScreenState, en lugar del bloque "Acordeón de conceptos"`

```
Padding(
  padding: EdgeInsets.symmetric(horizontal: sidePad),
  child: Column(children: [
    for (var i = 0; i < _concepts.length; i++) ...[
      _ConceptCard(concept: _concepts[i], index: i),
      const SizedBox(height: 56),
    ],
  ]),
),
```

Y envolver todo el `SingleChildScrollView` de la página en `ClipRect`, para que una tarjeta a medio entrar no cree scroll horizontal en web. Borrar `_openConcept`.

5. Apariciones, hover y lista de algoritmos

5.1 Widget nuevo: `Pressable` (hover en web, toque en móvil)

`lib/widgets/ui/pressable.dart (archivo nuevo)`

Con mouse, el elemento se eleva al pasar el cursor; con el dedo, se encoge al presionar. No se pregunta la plataforma: las pantallas táctiles no envían eventos de hover. `builder` entrega `hovered` para que la tarjeta pueda mostrar su descripción al pasar el mouse.

```
import 'package:flutter/material.dart';

class Pressable extends StatefulWidget {
  final VoidCallback? onTap;
  final double hoverLift;   // px hacia arriba con hover: 3 botones, 10 tarjetas
  final double pressScale;  // tamaño al presionar
  final Widget? child;
  final Widget Function(BuildContext context, bool hovered)? builder;
  const Pressable({super.key, this.onTap, this.hoverLift = 3, this.pressScale = 0.96,
      this.child, this.builder}) : assert(child != null || builder != null);

  @override
  State<Pressable> createState() => _PressableState();
}

class _PressableState extends State<Pressable> {
  bool _hover = false, _down = false;

  @override
  Widget build(BuildContext context) {
    final still = MediaQuery.disableAnimationsOf(context);
    final enabled = widget.onTap != null;
    final content = widget.builder?.call(context, _hover) ?? widget.child!;
    return MouseRegion(
      cursor: enabled ? SystemMouseCursors.click : MouseCursor.defer,
      onEnter: (_) => setState(() => _hover = true),
      onExit: (_) => setState(() => _hover = false),
      child: GestureDetector(
        onTapDown: enabled ? (_) => setState(() => _down = true) : null,
        onTapUp: enabled ? (_) => setState(() => _down = false) : null,
        onTapCancel: () => setState(() => _down = false),
        onTap: widget.onTap,
        child: AnimatedContainer(
          duration: still ? Duration.zero : const Duration(milliseconds: 250),
          curve: Curves.easeOutCubic,
          transform: Matrix4.translationValues(0, _hover && enabled ? -widget.hoverLift : 0, 0),
          child: AnimatedScale(
            scale: _down ? widget.pressScale : 1,
            duration: still ? Duration.zero : const Duration(milliseconds: 150),
            child: Semantics(button: enabled, child: content),
          ),
        ),
      ),
    );
  }
}
```

5.2 Widget nuevo: `RevealOnce` (aparece una sola vez)

`lib/screens/home/reveal_once.dart (archivo nuevo)`

Para los títulos de sección y las tarjetas de la lista horizontal: entran deslizándose la primera vez que se ven y se quedan.

```
import 'package:flutter/material.dart';
import 'package:visibility_detector/visibility_detector.dart';

class RevealOnce extends StatefulWidget {
  final String id;        // único en la página: 'titulo-algoritmos', 'metodo-3'...
  final Offset from;      // de dónde entra, en fracciones del tamaño: (0, 0.3) abajo, (0.6, 0) derecha
  final Duration delay;   // para escalonar la lista
  final Widget child;
  const RevealOnce({super.key, required this.id, this.from = const Offset(0, 0.3),
      this.delay = Duration.zero, required this.child});

  @override
  State<RevealOnce> createState() => _RevealOnceState();
}

class _RevealOnceState extends State<RevealOnce> {
  bool _shown = false, _scheduled = false;

  void _onVisible(VisibilityInfo info) {
    if (_scheduled || info.visibleFraction < 0.2) return;
    _scheduled = true;
    Future.delayed(widget.delay, () { if (mounted) setState(() => _shown = true); });
  }

  @override
  Widget build(BuildContext context) {
    if (MediaQuery.disableAnimationsOf(context)) return widget.child;
    const d = Duration(milliseconds: 900);
    return VisibilityDetector(
      key: Key('reveal-${widget.id}'),
      onVisibilityChanged: _onVisible,
      child: AnimatedSlide(
        offset: _shown ? Offset.zero : widget.from,
        duration: d, curve: Curves.easeOutCubic,
        child: AnimatedOpacity(opacity: _shown ? 1 : 0, duration: d, child: widget.child),
      ),
    );
  }
}
```

`lib/screens/home.dart → títulos "Antes de empezar" y "ALGORITMOS Y ORDENAMIENTOS"`

```
RevealOnce(id: 'titulo-algoritmos', child: Text('ALGORITMOS Y ORDENAMIENTOS', ...)),
```

5.3 Lista horizontal de algoritmos

`lib/screens/home.dart → bloque "Tarjetas de métodos" y class _MethodCard`

1. **Entrada escalonada:** cada tarjeta dentro de `RevealOnce(id: 'metodo-$i', from: const Offset(0.6, 0), delay: Duration(milliseconds: 90 * i), ...)`.

2. **Hover y toque:** la tarjeta usa `Pressable(hoverLift: 10, onTap: onTap, builder: (context, hovered) => ...)`. La descripción y el botón «Abrir» se muestran si `isOpen || hovered`: en web aparecen al pasar el mouse, en móvil al tocar. El `InkWell` actual se quita (Pressable ya maneja el toque).
3. **Ícono +/×:** en vez de cambiar entre `Icons.add` e `Icons.remove`, girar el mismo ícono: `AnimatedRotation(turns: open ? 0.125 : 0, child: Icon(Icons.add))`.
4. **Arrastre con mouse en web:** Flutter ignora por defecto el arrastre con mouse en listas horizontales. Envolver la lista:

```
// import 'package:flutter/gestures.dart';
class _DragWithMouse extends MaterialScrollBehavior {
  @override
  Set<PointerDeviceKind> get dragDevices => {...super.dragDevices, PointerDeviceKind.mouse};
}

ScrollConfiguration(
  behavior: _DragWithMouse(),
  child: SingleChildScrollView(scrollDirection: Axis.horizontal, ...),
)
```

6. Fuera de esta tarea

- No tocar `main.dart` más allá de lo indicado en la sección 2.
- No cambiar otras pantallas (editor, Johnson, Asignación, ordenamientos): eso es el marco compartido (pasos 4–9 de la guía) y lo hace otra persona.

7. Lista de verificación para entregar

- [ ] `main.dart` no fuerza tema oscuro ni la fuente `.SF Pro Text`; las demás pantallas se ven como antes.
- [ ] Buscar `Color(0x` en `home.dart` no devuelve resultados; todos los colores vienen de `AppColors`.
- [ ] No queda el acordeón ni `_openConcept`.
- [ ] Las cuatro tarjetas de concepto entran con el scroll: 01 y 03 por la izquierda, 02 y 04 por la derecha, y se detienen al verse completas.
- [ ] Al subir, las tarjetas vuelven a salir por su lado.
- [ ] Cada tarjeta tiene número, título, texto, imagen y botón «Ver en YouTube» que abre el enlace.
- [ ] En un teléfono (menos de 700 px) la imagen va arriba y el texto debajo.
- [ ] Los títulos de sección y la lista de algoritmos aparecen una sola vez, la lista de forma escalonada.
- [ ] En web, pasar el mouse sobre una tarjeta de algoritmo la eleva y muestra su descripción; en móvil, tocarla la muestra.
- [ ] La lista horizontal se puede arrastrar con el mouse en web.
- [ ] Con «reducir movimiento» activado en el dispositivo, todo aparece sin animaciones.
- [ ] No hay scroll horizontal de la página en web mientras las tarjetas entran.
- [ ] `flutter analyze` no muestra errores ni avisos nuevos en los archivos tocados.